Symbolic Substrates:
VSL as a Viable Coordination Medium Between Heterogeneous Cognitive Systems

Luis Pineda R.*

Viable System Framework Project

Draft skeleton — June 2026

Abstract

We describe VSL (Viable System Language), a domain-specific language derived from Beer's Viable System Model, and ask of it two questions that turn out to be one. The first is what VSL *is*: a notation simultaneously parseable by a deterministic grammar, generatable by a cognitive agent (a large language model), and semantically precise enough to govern a multi-layer operational system. We argue that an agent generating VSL is not translating but *compressing* — extracting operational intent from natural language and encoding it in the minimum formal representation that preserves that intent. This *agent-as-compiler* framing makes language quality measurable: parse rate, semantic fidelity, round-trip preservation.

The second question is whether VSL *works* as a coordination substrate between two *different* minds. Here a parse guarantee is necessary but not sufficient: when one agent encodes intent and a heterogeneous agent decodes it, a correct parse tree with the wrong referents is a silent failure. We measure this directly and find a sharp split — *structural* fidelity crosses at $\approx 100\%$ while *semantic* fidelity drifts to $\approx 40\%$, the receiving model substituting its own operational context for the sender's. A lightweight domain anchor (≈ 8 tokens) attenuates the drift; but the load-bearing result is architectural. By partitioning the message into *computed* fields (executed deterministically by a wrapper) and *interpreted* fields (executed by the agent), fidelity on the computed axis becomes 1.0 by construction — Ashby's Law of Requisite Variety applied at the level of the grammar, with computational variety routed to a deterministic layer and semantic variety to the cognitive one. Meaning survives the crossing not because the model is large enough, but because the substrate stops asking the model to do what only a parser can.

We report compression (38–79% against natural language, growing with protocol complexity), the structural/semantic drift and its anchor, the computed/interpreted

*Contact: luis@tallerpineda.cl

split (computed fidelity = 1.0 across five tasks; 18× message compression), a capability taxonomy whose boundary is domain knowledge rather than task type, and the substrate's measurable *cost of existing*. We close with the information-theoretic reading: the grammar is a codebook and the set of valid VSL files is a low-dimensional manifold in string space, so generating VSL is finding a point on that manifold.

Keywords: viable system model; domain-specific language; LLM-as-compiler; semantic compression; requisite variety; second-order cybernetics; coordination substrate; Shannon entropy; VSL.

Contents

1 The Problem of Saying What a System Is **3**

2 A Grammar Derived From Structure, and the Agent That Compiles It **3**

2.1 The agent as compiler 3

3 Does Meaning Survive the Crossing? **4**

4 Requisite Variety at the Grammar Level **4**

5 The Cost of Depending on Another Mind **5**

6 Measuring the Language **6**

6.1 Compression 6

6.2 Parse rate and generation 6

6.3 Shannon entropy of VSL versus natural language 6

7 Grammar as a Mirror of the Architecture **7**

8 What a Substrate Is For **7**

1 The Problem of Saying What a System Is

[TO WRITE — origin story: the first session handoff that was too long for a context window but too structured for prose; the gap was not accidental.]

Four candidate media each fail in a specific way: natural language is ambiguous and verbose (ten to fifty times the tokens of the state it describes, with no parse guarantee); JSON/YAML parse but are semantically flat (no notion of system layer, channel type, or viability constraint); general-purpose code is Turing-complete but domain-blind (a struct does not know it is an S3 node); and mathematics is precise but not generatable in real-time interaction. The S2 coordination channel between cognitive agents had no adequate medium.

[TO WRITE — develop each failure mode; introduce the deeper, second-order problem: even a medium that parses may not carry meaning across two different minds.]

2 A Grammar Derived From Structure, and the Agent That Compiles It

VSL expresses three things and deliberately omits the rest: system identity $\texttt{V}(id, L)$, channel contracts ($\varphi\uparrow$, $\varphi\downarrow$, algedonic), and viability state (`viable`, `blocked`, `partial`). Its recursion mirrors the VSM's: any S1 unit may embed a full system.

```
@vsm 0.1
V(vsf-nucleus, L1) {
  status: viable
  channel: phi_up -> V(vsf-s3-star, L2)
  channel: phi_down <- V(vsf-s3, L2)
  channel: algedonic -> V(vsf-s5, L3)
  recursive: false
}
```

[TO WRITE — the BNF fits a page; every production has a reason; what VSL omits (implementation detail, code, prose) is a design decision, not a limitation.]

2.1 The agent as compiler

A classical compiler maps source to machine code: deterministic, lower abstraction, no ambiguity resolution, correctness binary. An agent generating VSL maps natural language (high ambiguity, low precision, high expressiveness) to VSL (low ambiguity, high precision, constrained expressiveness). This is not classical compilation but *semantic compression*:

the agent extracts operational intent and encodes it in the minimum formal representation that preserves it. Its quality is therefore a distribution — parse rate, semantic fidelity, round-trip preservation — not a bit. The constraint that makes this clean is D6 (no language model at L0/L1): the agent operates at $L \geq 2$, generating VSL that governs L0/L1 processes. The cognitive/operational boundary *is* the language boundary. In our architecture Qwen acts as S1, Claude as S3–S5, and VSL is the S2 channel between them.

[TO WRITE — develop agent-as-compiler vs classical compiler; the measurable-quality claim sets up the experiments of Sections 3–6.]

3 Does Meaning Survive the Crossing?

A parse guarantee is necessary but not sufficient. When agent A encodes intent as VSL and a *heterogeneous* agent B decodes it, the question is whether B reconstructs A's operational referents or its own. A correct parse tree with the wrong referents is a *silent* failure — the second-order problem (the cybernetics of observing systems) made concrete.

We measured it. Submitting schema-mode VSL to an independent model with the VSF schema preloaded, structural fidelity — correct S-layer hierarchy, $\varphi\uparrow/\varphi\downarrow$ channels, PSI thresholds, the covenant step, metric names — was $\approx 100\%$. Semantic fidelity was $\approx 40\%$: the model systematically substituted its own context for VSF referents ($\varphi\uparrow$ checkpoints mapped to cloud-sync; `vm-nucleus` to an internal knowledge module). The single notion of "fidelity" had conflated two independent properties.

Property	Crosses the substrate at	Failure mode
Structural fidelity	$\approx 100\%$	—
Semantic fidelity (no anchor)	$\approx 40\%$	context substitution (silent)
Semantic fidelity (with anchor)	$\approx 95\%$	—

Table 1: Structural vs. semantic fidelity across a heterogeneous crossing (EXP-PHI-01). Structure is reconstructed almost perfectly; meaning drifts unless anchored. A `domain_context` header of ≈ 8 tokens attenuates the drift from $\approx 60\%$ to $\approx 5\%$.

```
domain_context: {target=vsf-nucleus, env=Rust/LXC, layer=S1}
```

[TO WRITE — develop context substitution as a structural property of heterogeneous coupling, not a model defect; the anchor as a cheap variety attenuator; set up the deeper fix.]

4 Requisite Variety at the Grammar Level

The anchor attenuates the drift; it does not eliminate the cause. The cause is that we were asking the cognitive layer to perform computational work — counting, matching,

enumerating — at which a parser is exact and a language model is merely probable. The fix is Ashby's: route variety to the layer competent to handle it. We partition the message's `output_format` into *computed* fields, executed deterministically by a wrapper before the agent is ever called, and *interpreted* fields, executed by the agent alone.

```
output_format: {
  computed: { line_count: INTEGER, token_frequencies: OBJECT, regex_matches:
     ARRAY }
  interpreted: { summary: STRING, key_themes: ARRAY, cybersyn_mapping: OBJECT }
}
```

On the computed axis, fidelity is 1.0 by construction: there is no model variance to contend with, because no model is consulted. On the interpreted axis, fidelity remains capability-dependent — which is correct, because that is where genuine interpretation lives.

Axis	Fidelity	Mechanism
Computed (wrapper)	1.00 (5/5 tasks, 3 runs)	deterministic by design
Interpreted (agent)	capability-dependent	domain knowledge of the model
Message size (split)	$\approx$ 450 B vs. 8000 B	18$\times$ compression

Table 2: The computed/interpreted split (EXP-PHI-02). Partitioning by competence makes computed fidelity a design invariant rather than a model outcome, architecturally resolving the closure-is-not-correctness problem of EXP-PHI-01. The latency target was not met under a 1.5 B model on CPU — a hardware/model bound, not a property of the split.

This architecturally closes the earlier finding that *loop closure is not correctness*: the failure was structural, not a deficit of model scale. A capability taxonomy follows naturally — and its boundary is instructive. Reliability on interpreted fields tracks the model's *domain knowledge*, not the task type: a 1.5 B model correctly summarised an ethics specification but read "covenant" as a legal contract, and mapped a governance `watchdog` to S3 (control) rather than S5 (policy). The boundary that matters is not counting-vs-reasoning but in-domain-vs-out.

[TO WRITE — develop the Ashby framing as the paper's central constructive result; the capability matrix; the two boundaries (task-type robust, domain-knowledge model-specific).]

5 The Cost of Depending on Another Mind

A substrate that depends on an external cognitive agent is not free. Its cost of existing is multi-dimensional — credentials, routing, upstream availability, dynamic resources —

and it is structural, surfacing in the loop rather than as an accident of any one run. This connects the substrate to the broader cost-of-coherence thread.

[TO WRITE — develop H5; the sovereignty cost as a measurable property; the bridge to the epistemic-viability companion paper.]

6 Measuring the Language

6.1 Compression

Case	T_A	$T_{B,\text{full}}$	$T_{B,\text{schema}}$	r_{full}	r_{schema}
`pvsnp`	328	161	—	0.51	—
`thermal_senescence`	638	346	—	0.46	—
`spec_boot+Lyapunov`	717	432	148	0.40	0.79
`proxmox+S4`	698	411	198	0.41	0.72
`PSI/1.0`	641	397	147	0.38	0.77

Table 3: VSL compression (tiktoken `cl100k_base`). Standalone VSL achieves 38–51%; with a shared schema (machine-to-machine), 72–79%, growing with protocol complexity. r is the compressed-to-original token ratio.

6.2 Parse rate and generation

The VSL v1.0 parser accepts 212/217 corpus files ($\approx$ 97.7%); the five failures are legacy-format edge cases, not grammar violations. An S1 agent (Qwen) generates valid `next_experiment.vsm` documents through a plan/eval/propose loop, the operational demonstration that the grammar is learnable from the specification alone.

6.3 Shannon entropy of VSL versus natural language

EXP-VSL-04 measured Shannon entropy over five paired specifications — each written once in natural-language prose and once in VSL with equivalent operational content — at two granularities: character-level, and token-level under a production tokenizer (`cl100k_base`). The two levels dissociate, and the dissociation is the finding. At the character level VSL carries *more* entropy than prose (H_{char}: 5.52 vs. 4.99 bits, +10.6%): the notation's symbol inventory (`@`, `:=`, braces, arrows) widens the character distribution. At the token level the order reverses (H_{tok}: 6.89 vs. 7.28 bits, -5.4%, holding in all five pairs): the grammar constrains how symbols combine, so the token stream is more predictable than prose even though the character stream is less so. The codebook operates at the token level — the level at which an agent consumes the specification. Combined with the token-count

compression of Table 3 (38–79% fewer tokens for the same content), lower per-token entropy over fewer tokens is the information-theoretic signature of a constrained grammar doing real coding work.

Two boundaries on the claim. First, this is evidence of a *good* code, not an optimal one: optimality against the source entropy of "viable-system specifications" would require a rate floor we cannot estimate, and "semantic entropy" has no agreed definition we are willing to build on. The operational proxy we adopt instead is task-conditional: a code is good to the extent that a competent agent reconstructs the specified system from fewer tokens at equal fidelity — Section 3's experiment restated in coding-theoretic terms. Second, the schema-only variants compress further but raise character entropy further (+0.18 to +0.67 bits over prose): stripping inline definitions concentrates notation, a rate–redundancy trade the grammar makes visible rather than hides.

7 Grammar as a Mirror of the Architecture

Every gap in VSL v1 corresponds to an architectural debt; VSL v2 closes those gaps in parallel with the system closing them. A rejected-$\varphi\downarrow$ channel appears once S1 validates $\varphi\downarrow$ before applying it; a `predictive_alert` event and an `env_sample` channel appear once the system has an external environment to sense (the companion kernel paper shows the internal anticipatory channel is empty without external variance, motivating exactly this grammar); a `veto` channel, distinct from the algedonic alert, appears once S5 can halt as well as warn. The grammar does not run ahead of the architecture — it is precisely aligned with it, and its honesty about what it cannot yet express is part of the contribution.

The mirror thesis yields one forward-looking, falsifiable prediction. The architecture is recursive — any S1 unit may itself be a complete VSM — and if the grammar truly mirrors the architecture, the *specification* should inherit that self-similarity: the compression ratio $r_L = |V_L|/|V_{L-1}|$ between a level-L spec and the specs of its level-$(L-1)$ constituents should be approximately constant across recursion levels. A two-level recursive deployment (planned) tests this directly. We state it as a prediction about description length only; we make no claim that this ratio relates to the dynamical contraction rate of the kernels themselves, which lives in a different space.

[TO WRITE — develop grammar-as-architecture-mirror; v2 as the documented co-evolution; keep the claims carried by v1 (H1/H3/H5/H6/H7/H8/H9) distinct from v2 designed extensions.]

8 What a Substrate Is For

A language is successful when it compresses the right things. VSL compresses operational state; S5 policy deliberately remains in natural language, because policy carries an

intentionality VSL is not designed to encode — the boundary is a choice, not a defect. The grammar is a codebook and valid VSL is a low-dimensional manifold in the space of all strings; the agent that generates VSL is finding a point on that manifold, and the substrate is viable precisely when the manifold is reachable by a cognitive agent and the computational variety has been routed elsewhere.

[TO WRITE — closing argument; open questions: emergent language (can an agent learn VSL without being taught?); can VSL express its own cost-of-existing?]

References

[1] Beer, S. (1972). *Brain of the Firm*. Allen Lane.

[2] Beer, S. (1979). *The Heart of Enterprise*. Wiley.

[3] Ashby, W. R. (1956). *An Introduction to Cybernetics*. Chapman & Hall.

[4] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27.

[5] von Foerster, H. (1974). *Cybernetics of Cybernetics*. BCL, University of Illinois.

[6] Medina, E. (2011). *Cybernetic Revolutionaries*. MIT Press.